

A Pipeline for Visualising MATSim Events in Blender

Daniel Steven Bell, Newcastle University (UK)

daniel.bell2@ncl.ac.uk

Abstract

Blender is a well-established open-source 3D modelling and animation tool. It benefits from a robust suite of functionality, highly capable render engines bundled as default, an active community, and an extensive catalogue of add-ons. Blender is also increasingly used for visualising scientific and simulation data, though this typically requires a dedicated pipeline to convert raw outputs into a Blender-compatible format. This is also true for MATSim outputs, which are not directly compatible with existing GIS tools in the workflow, particularly when spatial referencing must be preserved. Developing a method for importing MATSim outputs into Blender would significantly expand the visualisation and presentation options available to MATSim users. This work presents an initial pipeline for transforming MATSim event data into a format that is compatible with Blender, spatially aligned with other GIS layers, three-dimensional, and animatable. It is demonstrated on the outputs of a 24-hour MATSim simulation of Newcastle upon Tyne, UK.

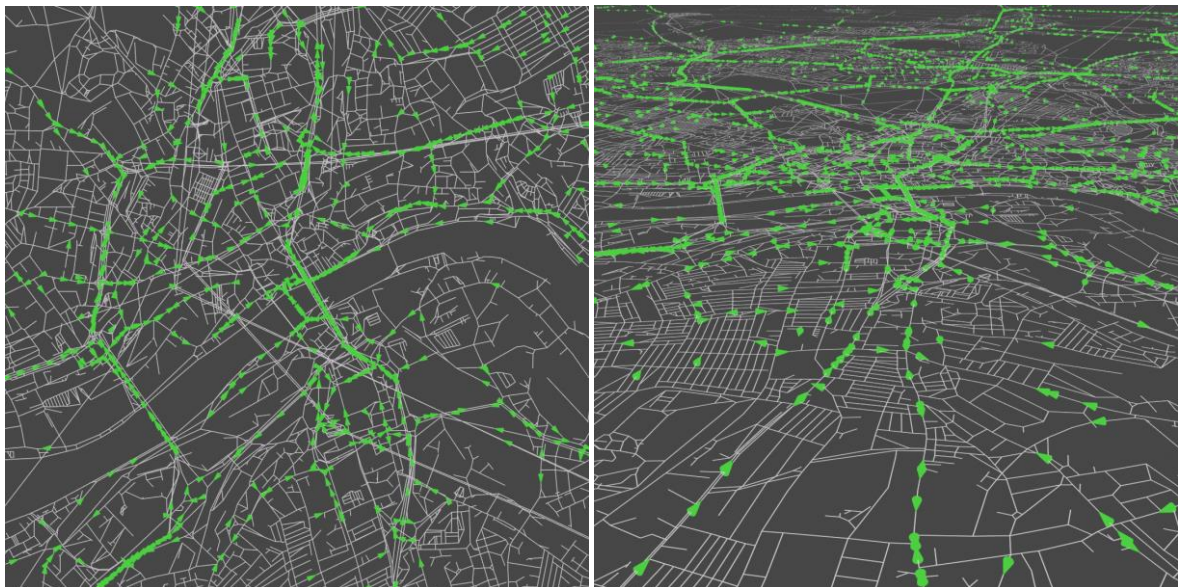


Figure 1: Renders of vehicles (green) and Newcastle upon Tyne road and rail network (grey). Left – orthographic, top-down view. Right – a perspective view of the same scene.

Introduction

Effective visualisation is central to how transport simulation results are interpreted, communicated, and acted upon, whether for technical debugging, model validation, or engaging non-technical stakeholders such as planners and the public. MATSim produces highly granular outputs describing the movement of individual agents over time. A range of tools exist for visualising MATSim outputs, such as SimWrapper (Charlton & Sana, 2023) and Via (Simunto), which provide strong support for interactive, web-based analysis of aggregate and disaggregate simulation results. However, these tools are not designed for the kind of immersive or cinematic 3D presentation this work explores. General-purpose 3D animation software has seen growing use for visualising scientific and simulation data more broadly, suggesting an opportunity to bring this style of presentation to transport simulation specifically (Lan et al., 2021; Marín et al., 2025). Realising this in practice requires bridging two ecosystems which were not designed to interoperate – that of MATSim's XML-based event and network data, and a 3D environment's expectations around geometry, coordinate systems, and animation. This work addresses that gap by developing and documenting an initial pipeline for transforming MATSim output into an animated, spatially accurate 3D scene, demonstrated on a real-world simulation case study.

Methodology

The pipeline from event data to rendering the scene consists of five stages (excluding running MATSim) (Figure 2). Two of these stages (Baking Vehicle Motion and Vehicle Motion Loader) are currently automated, with the other three having the possibility of automation or single setup, given some extra configuration. Test data used is from a 24-hour MATSim simulation of Newcastle upon Tyne, UK, containing 319,841 unique links, 125,706 unique nodes, 142,272 unique vehicles, and five modes (public transport, car, car_passenger, walk, and bike).

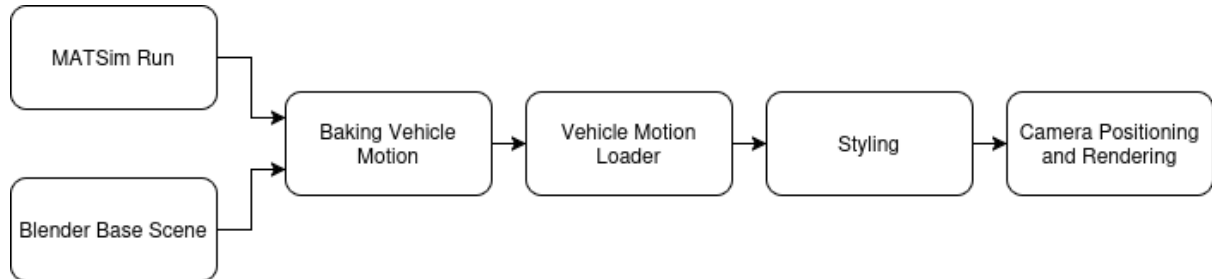


Figure 2: Working Pipeline

Scene Setup

The initial step is to create and save to disk a template blender file, which we will call *base.blend*. The purpose of this is to have an existing scene in which we can utilise the BlenderGIS (domlysz) add-on to import node positions into Blender and embed a script to load vehicle motions, as described in the following sections. Creating such a file also offers an opportunity to template preferred scene, styling, and render settings. In this case, a buffered network shapefile has been added to the scene using BlenderGIS. It should be ensured that the BlenderGIS add-on is installed and enabled within this file. Testing was performed on Blender version 5.2.0 LTS and BlenderGIS version v2.2.15, running on Ubuntu 24.04.4 LTS.

Baking Vehicle Motion

A Python script named *bake_vehicle_motion.py* is executed in order to transform the events described in *output_events.xml* into a single point cloud entity within Blender. This is performed on the *base.blend* scene while running Blender in headless mode. It is possible to execute python scripts from within the Blender UI in a dedicated scripting environment. However, the purpose for running Blender in headless mode at this stage is twofold. Firstly, it allows for access to system/environment python packages which cannot ordinarily be accessed within Blender. Secondly, there are stages at which the total number of Blender objects temporarily approach or exceed the limits of what Blender can display through its viewport, in some instances causing the program to crash.

Node data from *network.xml* act as waypoints for events. Node positions are read from the *network.xml* file and converted into a shapefile of points, which are then automatically imported to Blender as individual objects using BlenderGIS. BlenderGIS reprojects the shapefile from its source Coordinate Reference System (CRS) into Blender's local scene coordinate system, defined relative to a fixed origin rather than a standard geodetic or projected CRS. The benefit of using BlenderGIS for this step, rather than retaining the node data's original CRS or applying a custom transformation, is that node positions end up aligned within the same local coordinate system as other geodata imported through BlenderGIS, such as basemaps and SRTM elevation data. These node objects cannot remain within Blender – each point acts as an individual entity and causes strain on Blender's dependency graph. The new coordinate position of each node point is extracted and written to a numpy array, before removing all node points from the scene.

The next step involves parsing the *output_events.xml* file, baking raw simulation data to disk, and setting up an empty “skeleton” object within the Blender scene ready for animation. Using *event_reader* function from *matsim* python module, event types matching *vehicle enters traffic*, *vehicle leaves traffic*, *entered link*, or *left link* are extracted to build a numpy array (written to disk) of every vehicle event at every timestamp. Event link IDs are matched to corresponding nodes within *network.xml*, and in turn referenced to the new node positions calculated in the previous step. In setting up the animation, an empty mesh object consisting of one vertex per vehicle is created as a placeholder at the origin of the scene, as well as two custom attributes on this mesh called *visibility* and *direction*.

The final step of *bake_vehicle_motion.py* is to embed a separate program, a python script named *loader.py*, into the Blender project. The Blender file is then saved to disk under a new name.

Vehicle Motion Loader

Upon opening the newly created Blender file, the user is prompted to run *loader.py* from within Blender – an action which can be automated by modifying Blender settings. The loader script reads the numpy array containing vehicle motions from disk, and computes, for any given frame number: where every vehicle should be positioned, which direction it should face (based on direction of travel), and whether it should be visible at that specific frame. The function which performs this is registered as a frame handler, meaning that every time a frame is selected (during playback, scrubbing, or rendering), this function will be called and the *VehiclePoints* mesh created earlier will be updated with vehicle positions for that frame. F-curves are typically used within blender to interpolate an object position per frame between two waypoints. As all vehicles exist as only instances within a single point cloud object, F-curves cannot be utilised in this case, and so *loader.py* computes these positions on the fly.

Styling

Despite all of the setup and processing there remains a problem – *VehiclesPoints* mesh is not visible within Blender’s viewport. To remedy this, some simple styling using Blender tooling is required. Each vehicle instance needs to take on the form of a geometry. A mesh such as a cube, sphere, or cone can be added manually to the scene to act as a reference object in Geometry Node Editor (Figure 3). A material should also be added to this reference object so that vehicle instances will be visible at render time in the absence of any global lighting. The simplest way to do this is to add a Principled BSDF shader to the reference object and edit emission hue and strength values so that vehicle instances emit their own light source.

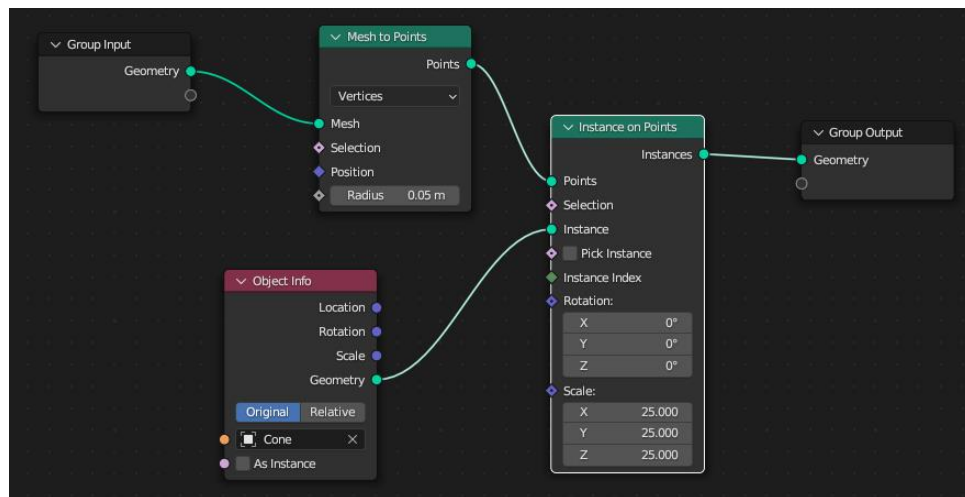


Figure 3: The most basic Geometry Node setup required to display *VehiclePoints*. In this case, a Cone object is used as a reference geometry for each instance.

Camera Positioning and Rendering

Camera positioning and frame selection is a task best undertaken manually after inspecting the scene, to ensure intended locations and periods are captured. For most scenes of this kind where processing speed is preferred over a highly realistic output aesthetic, built in render engine EEVEE is recommended. As this test simulation is over a period of 24-hours, there are a total of 86,400 frames. When it comes to rendering, these can easily be reduced with options frame start, frame end, step, and frame rate.

Conclusion

This work has presented an initial pipeline for transforming MATSim outputs into an animated, spatially accurate 3D scene within Blender, addressing the compatibility gap between MATSim's event-based output and existing

GIS and visualisation tooling. Node positions from network.xml were imported and reprojected using the BlenderGIS add-on, aligning them within a shared local coordinate system alongside other geodata in the form of a buffered network shapefile. Vehicle events from output_events.xml were then baked to disk and reconstructed within Blender as a per-frame animation, driven by a custom frame handler rather than standard F-curve interpolation, with simple geometry and shader-based styling applied to make vehicle instances visible at render time. This represents a first attempt at bridging MATSim and Blender, and a number of directions remain for future work: distinguishing between travel modes, styling vehicles based on velocity, improving the overall automation of the pipeline to reduce manual setup, and incorporating additional GIS layers such as basemaps and building geometry to produce a more complete and immersive scene.

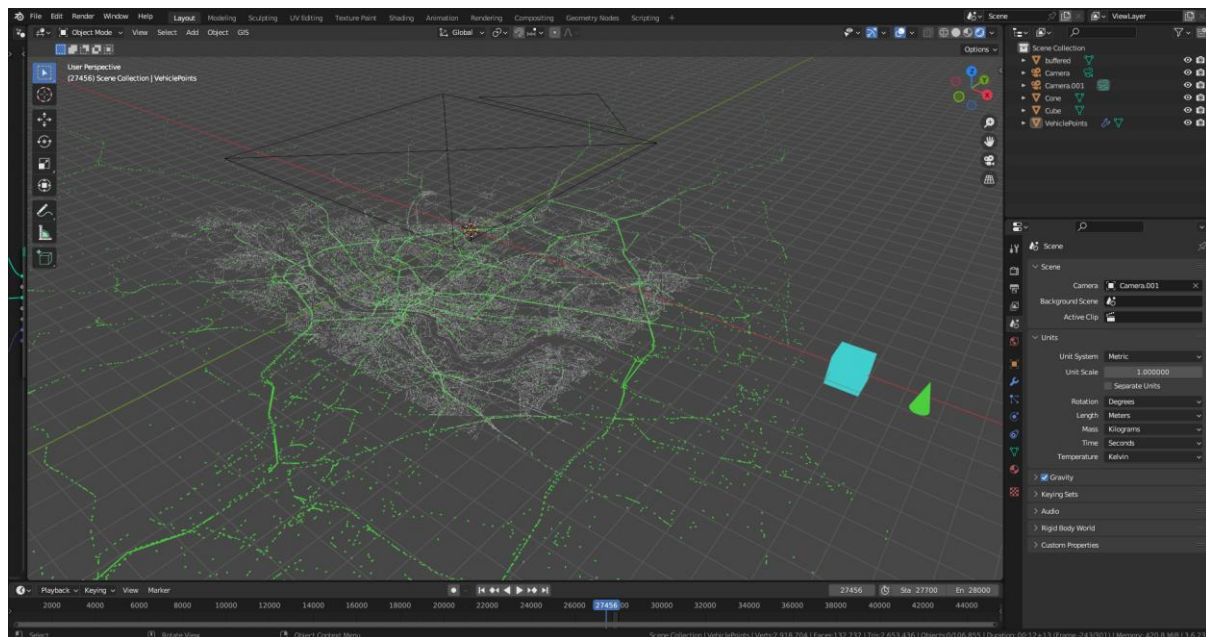


Figure 4: Blender scene screenshot, depicting cube and cone reference objects (right in viewport), Newcastle upon Tyne road and rail network (grey), vehicle instances (small green points), and top-down camera placement (square black outline).

References

- Charlton, W., & Sana, B. (2023). SimWrapper, an open source web-based platform for interactive visualization of microsimulation outputs and transport data. *Procedia Computer Science*, 220, 724–729. <https://doi.org/10.1016/j.procs.2023.03.095>
- Simunto, "Via," Simunto, [Online]. Available: <https://www.simunto.com/via/>. Accessed: Aug. 15, 2026.
- Lan, F., Young, M., Anderson, L., Ynnerman, A., Bock, A., Borkin, M. A., Forbes, A. G., Kollmeier, J. A., & Wang, B. (2021). Visualization in astrophysics: Developing new methods, discovering our universe, and educating the earth. *Computer Graphics Forum*, 40(3), 635–663. <https://doi.org/10.1111/cgf.14332>
- Marín, J., Baptiste, T. M. G., Rodero, C., Williams, S. E., Niederer, S. A., & García-Fernández, I. (2025). SciBlend: Advanced data visualization workflows within Blender. *Computers & Graphics*, 130, 104264. <https://doi.org/10.1016/j.cag.2025.104264>
- domlysz, "BlenderGIS," GitHub repository, 2025. [Online]. Available: <https://github.com/domlysz/BlenderGIS>. Accessed: Aug. 15, 2026.